

EX No:1B	EDA – Data Science Frameworks (CRISPDM, SEMMA, KDD)

Aim:

To study Exploratory Data Analysis and apply the CRISP-DM, SEMMA, and KDD data science frameworks to a Daily Google Search Trends US dataset by performing data loading, data understanding, preprocessing, feature creation, classification, evaluation, and comparison of the three frameworks.

Algorithm:

1. Import pandas, numpy, and the required machine learning libraries.
2. Load the Daily Google Search Trends US dataset and inspect the first few records and dataset shape.
3. Clean the search-query data, create the Trend_Category target variable, generate numerical features, and handle missing feature values using mean imputation.
4. Encode the categorical target variable using LabelEncoder, standardize the feature values using StandardScaler, and split the dataset into training and testing sets.
5. Train a Random Forest Classifier model to predict the trend category and evaluate it using Accuracy.
6. Apply the CRISP-DM, SEMMA, and KDD frameworks to perform data preparation, modeling, assessment, and interpretation of the Google Search Trends dataset..

Program:

```
# =====
# 1. IMPORT LIBRARIES
# =====

!pip -q install kagglehub

import kagglehub
import pandas as pd
import numpy as np
import glob
import os

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder, StandardScaler
from sklearn.impute import SimpleImputer
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score

# =====
# 2. DOWNLOAD AND LOAD THE REAL KAGGLE DATASET
# =====

dataset_path = kagglehub.dataset_download(
    "adrianjuliusaluocho/daily-google-search-trends-us"
)
```

```

csv_files = glob.glob(
    os.path.join(dataset_path, "**", "*.csv"),
    recursive=True
)

df = pd.read_csv(csv_files[0])

print("Dataset:", csv_files[0])
print("Shape:", df.shape)
print("Columns:", df.columns.tolist())

display(df.head())

# =====
# 3. PREPARE THE REAL DATA
# =====

df.columns = (
    df.columns
    .str.strip()
    .str.lower()
    .str.replace(" ", "_", regex=False)
)

query_candidates = [
    c for c in df.columns
    if any(
        w in c
        for w in ["query", "search", "trend", "keyword", "term"]
    )
]

if not query_candidates:
    raise ValueError("Search-query column not found.")

query_col = query_candidates[0]

df[query_col] = (
    df[query_col]
    .astype(str)
    .str.strip()
    .str.lower()
)

print("Search-query column:", query_col)

display(df[[query_col]].head())

# =====
# 4. CREATE Trend_Category
# =====

def assign_category(q):

    q = str(q).lower()

    groups = {
        "Technology": [
            "ai",
            "artificial intelligence",
            "chatgpt",
            "python",
            "software",
            "iphone",
            "android",
            "google",
            "microsoft",
            "tesla",

```

```

        "technology",
        "tech",
        "robot"
    ],

    "Sports": [
        "football",
        "soccer",
        "nba",
        "nfl",
        "cricket",
        "tennis",
        "baseball",
        "basketball",
        "olympics",
        "match",
        "game",
        "championship",
        "player",
        "team"
    ],

    "Entertainment": [
        "movie",
        "film",
        "actor",
        "actress",
        "music",
        "song",
        "concert",
        "netflix",
        "youtube",
        "celebrity",
        "tv",
        "show",
        "album"
    ],

    "News": [
        "news",
        "president",
        "election",
        "government",
        "war",
        "politics",
        "minister",
        "court",
        "law",
        "protest"
    ]
}

for cat, words in groups.items():
    if any(w in q for w in words):
        return cat

return "General"

df["Trend_Category"] = df[query_col].apply(assign_category)

print("Derived target distribution:")

print(df["Trend_Category"].value_counts())

display(
    df[[query_col, "Trend_Category"]].head(10)
)

```

```

# =====
# 5. CREATE NUMERIC FEATURES
# =====

numeric_cols = (
    df.select_dtypes(include=np.number)
      .columns
      .tolist()
)

df["query_length"] = df[query_col].str.len()

df["word_count"] = (
    df[query_col]
      .str.split()
      .str.len()
)

feature_cols = list(
    dict.fromkeys(
        numeric_cols[:5]
        + ["query_length", "word_count"]
    )
)

X = df[feature_cols].copy()
y = df["Trend_Category"].copy()

print("Features:", feature_cols)

print(
    "X shape:",
    X.shape,
    "| y shape:",
    y.shape
)

# =====
# 6. CRISP-DM
# =====

# Encode target
le = LabelEncoder()

y1 = le.fit_transform(y)

# Handle missing values
imp = SimpleImputer(strategy="mean")

X1 = imp.fit_transform(X)

# Standardization
sc = StandardScaler()

X1 = sc.fit_transform(X1)

# Train-Test Split
Xtr, Xte, ytr, yte = train_test_split(
    X1,
    y1,
    test_size=0.2,
    random_state=42,
    stratify=y1
)

```

```

# Random Forest Classifier
model_crisp = RandomForestClassifier(
    n_estimators=100,
    random_state=42
)

model_crisp.fit(Xtr, ytr)

# Prediction
pred = model_crisp.predict(Xte)

# Evaluation
crisp_accuracy = accuracy_score(
    yte,
    pred
)

print(
    "CRISP-DM Accuracy:",
    round(crisp_accuracy, 4)
)

print(
    "CRISP-DM Accuracy (%) :",
    round(crisp_accuracy * 100, 2)
)

# =====
# 7. SEMMA
# =====

# Sample
Xs = df[feature_cols].copy()

ys = df["Trend_Category"].copy()

# Train-Test Split
Xtr, Xte, ytr, yte = train_test_split(
    Xs,
    ys,
    test_size=0.2,
    random_state=42,
    stratify=ys
)

# Encode target
les = LabelEncoder()

ytr = les.fit_transform(ytr)

yte = les.transform(yte)

# Modify - Imputation
imps = SimpleImputer(strategy="mean")

Xtr = imps.fit_transform(Xtr)

Xte = imps.transform(Xte)

# Standardization
scs = StandardScaler()

Xtr = scs.fit_transform(Xtr)

```

```

Xte = scs.transform(Xte)

# Model
model_semma = RandomForestClassifier(
    n_estimators=100,
    random_state=42
)

model_semma.fit(Xtr, ytr)

# Assess
semma_accuracy = accuracy_score(
    yte,
    model_semma.predict(Xte)
)

print(
    "SEMMA Accuracy:",
    round(semma_accuracy * 100, 2),
    "%"
)

# =====
# 8. KDD
# =====

# Selection
Xk = df[feature_cols].copy()

yk = df["Trend_Category"].copy()

# Encode target
lek = LabelEncoder()

yk = lek.fit_transform(yk)

# Preprocessing
impk = SimpleImputer(strategy="mean")

Xk = impk.fit_transform(Xk)

# Transformation
sck = StandardScaler()

Xk = sck.fit_transform(Xk)

# Data Mining - Train-Test Split
Xtr, Xte, ytr, yte = train_test_split(
    Xk,
    yk,
    test_size=0.2,
    random_state=42,
    stratify=yk
)

# Random Forest Classifier
model_kdd = RandomForestClassifier(
    n_estimators=100,
    random_state=42
)

model_kdd.fit(Xtr, ytr)

```

```

# Interpretation / Evaluation
kdd_accuracy = accuracy_score(
    yte,
    model_kdd.predict(Xte)
)

print(
    "KDD Accuracy:",
    round(kdd_accuracy * 100, 2),
    "%"
)

# =====
# 9. COMPARISON
# =====

comparison = pd.DataFrame({

    "Methodology": [
        "CRISP-DM",
        "SEMMA",
        "KDD"
    ],

    "Accuracy (%)": [
        crisp_accuracy * 100,
        semma_accuracy * 100,
        kdd_accuracy * 100
    ],

    "Main Focus": [
        "Complete project lifecycle",
        "Model building and assessment",
        "Knowledge discovery"
    ],

    "Strength": [
        "Comprehensive and industry standard",
        "Strong analytics workflow",
        "Good for research and preprocessing"
    ],

    "Limitation": [
        "Requires more documentation",
        "No deployment phase",
        "No deployment phase"
    ]
})

comparison["Accuracy (%)"] = (
    comparison["Accuracy (%)"]
    .round(2)
)

display(comparison)

# =====
# 10. ACCURACY VISUALIZATION
# =====

import matplotlib.pyplot as plt

plt.figure(figsize=(8, 5))

plt.bar(
    comparison["Methodology"],
    comparison["Accuracy (%)"]
)

```

```

)

plt.title(
    "Accuracy Comparison of CRISP-DM, SEMMA and KDD"
)

plt.xlabel("Methodology")

plt.ylabel("Accuracy (%)")

plt.ylim(0, 100)

for i, v in enumerate(
    comparison["Accuracy (%)"]
):

    plt.text(
        i,
        v + 1,
        f"{v:.2f}%",
        ha="center"
    )

plt.show()

```

Output:

```

s/174.archive...
100%|██████████| 935k/935k [00:00<00:00, 20.9MB/s]
Extracting files...
Dataset: /root/.cache/kagglehub/datasets/adrianjuliusaluocho/daily-google-search-trends-us/versions/194/trending_searches_in_us.csv
Shape: (31804, 8)
Columns: ['query', 'start_date', 'end_date', 'active', 'search_volume', 'increase_percentage', 'categories', 'trend_breakdown']

```

	query	start_date	end_date	active	search_volume	increase_percentage	categories	trend_
0	shedeur sanders	2026-03-30 18:40:00+00:00	NaN	True	200	50	Sports	
1	whoopi goldberg	2026-03-30 18:40:00+00:00	NaN	True	5000	50	Politics	
2	mo williams	2026-03-30 18:00:00+00:00	NaN	True	5000	300	Sports	ma baske
3	sloane stephens	2026-03-30 17:50:00+00:00	NaN	True	1000	200	Sports, Health	
4	canceled tv shows 2026	2026-03-30 17:50:00+00:00	NaN	True	5000	200	Entertainment	r cancel

Search-query column: query

query	
0	shedeur sanders
1	whoopi goldberg
2	mo williams
3	sloane stephens
4	canceled tv shows 2026

Derived target distribution:

Trend_Category

General 27477

Sports 1920

Technology 1171

News 776

Entertainment 460

Name: count, dtype: int64

	query	Trend_Category
0	shedeur sanders	General
1	whoopi goldberg	General
2	mo williams	General
3	sloane stephens	General
4	canceled tv shows 2026	Entertainment
5	atlas - guadalajara	General
6	karoline leavitt	General
7	kali uchis	General
8	germany vs ghana	General
9	caleb flynn	General

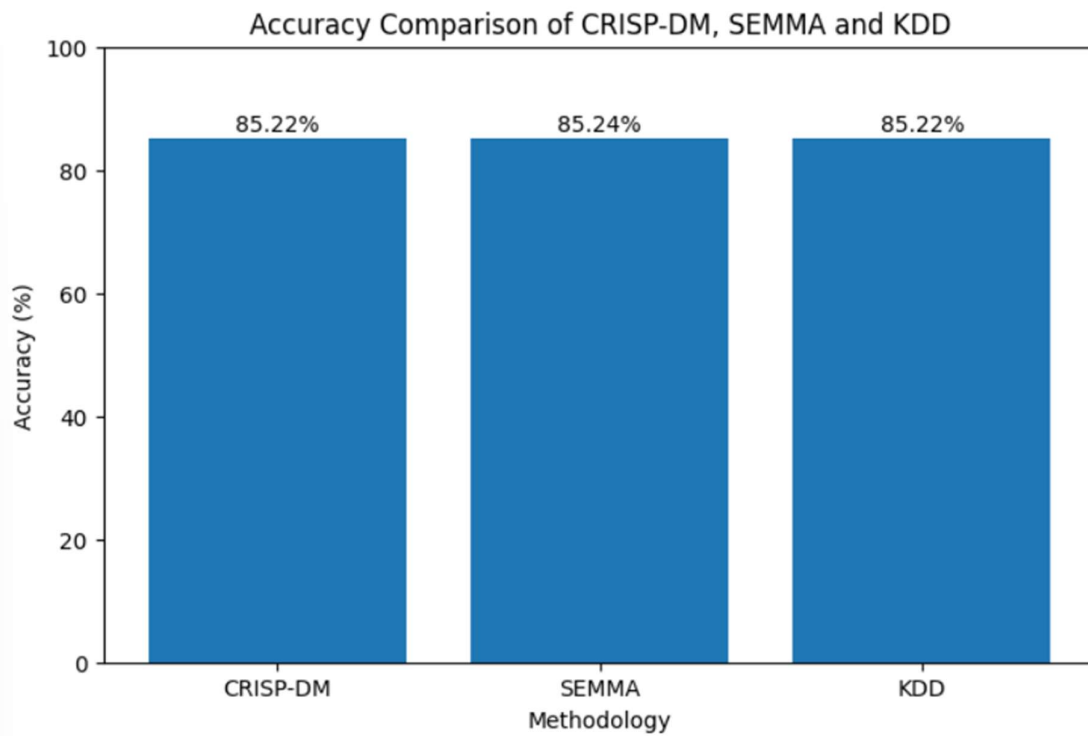
KDD Accuracy: 85.22 %

CRISP-DM Accuracy: 0.8522

CRISP-DM Accuracy (%): 85.22

SEMMA Accuracy: 85.24 %

	Methodology	Accuracy (%)	Main Focus	Strength	Limitation
0	CRISP-DM	85.22	Complete project lifecycle	Comprehensive and industry standard	Requires more documentation
1	SEMMA	85.24	Model building and assessment	Strong analytics workflow	No deployment phase
2	KDD	85.22	Knowledge discovery	Good for research and preprocessing	No deployment phase



Result: